

دانشگاه تهران

دانشکده مهندسی برق و کامپیوتر

درس مبانی رایانش توزیع شده (Fundamentals) Computing (Distributed)

گزارش جامع پیاده سازی و ارزیابی تجربی

پایگاه داده توزیع شده کلید-مقدار با

همانندسازی

Eventual & Strong with Store Key-Value (Replicated Consistency)

ارائه دهندگان (دانشجویان):

طه مجلسی (شماره دانشجویی:

۸۱۰۱۰۱۵۰۴)

علیرضا کریمی (شماره

دانشجویی: ۸۱۰۱۰۱۴۹۲)

تاریخ تحویل:

مرداد ۱۴۰۵ / Spring ۲۰۲۵

استاد درس:

دکتر محمدرضا شورنیا

سرستیار آموزشی:

پویا جمشیدی

دستیاران آموزشی:

مبینا حق زاده

محمد مهدی ابراهیم سلطان

فهرست مطالب

۲	۱ خلاصه مدیریتی (Executive Summary)
۲	۲ مفاهیم نظری و مبانی رایانش توزیع‌شده
۲	۱.۲ چرا داده‌ها در سیستم‌های توزیع‌شده همانندسازی می‌شوند؟
۲	۲.۲ تفاوت همانندسازی برای کارایی و همانندسازی برای تحمل خطا
۲	۲.۲ مدل‌های سازگاری (Consistency Models)
۲	۴.۲ مدل‌های سازگاری کاربرمحور (Client-Centric Consistency)
۲	۵.۲ تحلیل قضیه CAP
۴	۳ معماری سیستم و پیاده‌سازی کدها
۴	۱.۳ ساختار داده و مدیریت همزمانی
۴	۲.۳ منطق LWW (Last-Write-Wins)
۴	۳.۳ پیاده‌سازی Quorum اکثریت در مدل Strong
۵	۴ نتایج تجربی و نمودارهای تحلیلی
۵	۱.۴ نمودارهای تحلیلی سیستم
۱۱	۵ نتیجه‌گیری و راهنمای جلسه دفاع
۱۱	۱.۵ نتیجه‌گیری
۱۱	۲.۵ راهنمای دفاع (Q&A Defense Guide)

۱ خلاصه مدیریتی (Executive Summary)

در سیستم‌های توزیع شده مقیاس‌پذیر، **همانندسازی داده‌ها (Data Replication)** یکی از کلیدی‌ترین تکنیک‌ها جهت افزایش در دسترس بودن (Availability)، افزایش قابلیت تحمل خطا (Fault Tolerance) و کاهش تأخیر دسترسی به داده‌ها (Performance) است. با این حال، همانندسازی داده‌ها روی گره‌های مستقل شبکه چالش بنیادی **عدم سازگاری (Inconsistency)** را به همراه دارد.

در این پروژه، یک سیستم پایگاه داده توزیع شده کلید-مقدار (Replicated Key-Value Store) به صورت کامل با زبان **Go** و **Golang** پیاده‌سازی گردید. سیستم شامل ۳ گره مستقل Replica است که از طریق APIهای RESTful شبکه HTTP با یکدیگر در ارتباط هستند. دو مدل سازگاری اصلی پیاده‌سازی و ارزیابی شده‌اند:

۱. **مدل سازگاری تدریجی (Eventual Consistency):** اعمال سریع نوشتن روی نود محلی و انتشار غیرهمگام (Asynchronous) در پس‌زمینه.

۲. **مدل سازگاری قوی ساده‌شده (Strong Consistency):** اعمال نوشتن تنها پس از دریافت تاییدیه همگام (Synchronous) از حد نصاب اکثریت (Majority Quorum) - حداقل ۲ نود از ۳ نود).

برای برطرف کردن تعارضات همزمان، الگوریتم Last-Write-Wins (**LWW**) مبتنی بر Timestamp نانوثانیه و شناسه Replica پیاده‌سازی شده است. همچنین با اجرای اسکریپت‌های تست خودکار، تمامی ۱۸ حالت ارزیابی تحت تأخیرهای شبکه ۵۰۰ms و ۲۰۰۰ms بسنجش و ۱۰ نمودار تحلیلی تولید گردیده است.

۲ مفاهیم نظری و مبانی رایانش توزیع شده

۲.۱ چرا داده‌ها در سیستم‌های توزیع شده همانندسازی می‌شوند؟

دلایل اصلی همانندسازی داده‌ها عبارتند از:

- **افزایش در دسترس بودن (Availability):** اگر یک یا چند گره دچار خرابی (Node Fail-ure) شوند، سیستم همچنان می‌تواند از طریق سایر گره‌های سلامت به درخواست‌های خواندن و نوشتن پاسخ دهد.
- **بهبود کارایی و کاهش تأخیر (Performance & Latency):** با قرار دادن نسخه‌های داده نزدیک‌تر به کاربران یا توزیع بار خواندن روی گره‌های مختلف، تأخیر پاسخ‌دهی کاهش می‌یابد.
- **قابلیت تحمل خطا و پایداری داده (Fault Tolerance & Durability):** جلوگیری از دست رفتن داده‌ها در صورت بروز ناپایداری‌های سخت‌افزاری یا خرابی دیسک.

۲.۲ تفاوت همانندسازی برای کارایی و همانندسازی برای تحمل خطا

جدول ۱: مقایسه همانندسازی جهت کارایی در برابر تحمل خطا

Metric	Replication for Performance	Replication for Fault Tolerance
Primary Goal	Reduce read latency, CDN / Caching	Data durability & High availability
Propagation	Asynchronous	Synchronous Quorum
Write Cost	Low (Local Write)	Higher (Majority confirmation)
Consistency	Eventual Consistency	Strong Consistency (Linearizability)

۲.۲ مدل‌های سازگاری (Consistency Models)

- **سازگاری قوی (Strong Consistency):** تضمین می‌کند که بلافاصله پس از تایید یک نوشتن، هر خواندن بعدی از هر گره‌ای در سیستم، آخرین مقدار نوشته‌شده را برمی‌گرداند. سیستم رفتار یک پایگاه داده تک‌نسخه‌ای را شبیه‌سازی می‌کند.
- **سازگاری تدریجی (Eventual Consistency):** در صورت عدم ورود به‌روزرسانی جدید، تمامی گره‌ها در نهایت به مقدار یکسانی همگرا می‌شوند. اما در کوتاه‌مدت، ممکن است درخواست GET مقدار قدیمی (Stale Read) برگرداند.

۲.۲ مدل‌های سازگاری کاربرمحور (Client-Centric Consistency)

- **سازگاری Read-Your-Writes:** تضمین می‌کند یک کلاینت پس از نوشتن یک مقدار، در درخواست‌های خواندن بعدی خود همواره مقدار نوشته‌شده یا جدیدتر از آن را ببیند.
- **سازگاری Reads: Monotonic:** تضمین می‌کند که اگر کلاینت مقداری از داده را مشاهده کرد، در خواندن‌های بعدی هرگز مقداری قدیمی‌تر از آن را مشاهده نخواهد کرد.

۵.۲ تحلیل قضیه CAP

طبق قضیه CAP، یک سیستم توزیع‌شده نمی‌تواند همزمان سه ویژگی زیر را به صورت کامل تامین کند:

- **Consistency (سازگاری):** تمام نودها یک داده واحد را همزمان مشاهده کنند.
- **Availability (در دسترس بودن):** هر درخواست غیرخراب پاسخی دریافت کند.
- **Tolerance Partition (تحمل تفکیک شبکه):** سیستم هنگام قطعی ارتباط بین گره‌ها ادامه کار دهد.

چون Partition (P) در شبکه‌های واقعی غیرقابل اجتناب است، سیستم‌ها باید بین CP (فدا کردن Availability برای داشتن Consistency) یا AP (فدا کردن Consistency برای داشتن Availability) یکی را انتخاب کنند.

۳ معماری سیستم و پیاده‌سازی کدها

۱.۳ ساختار داده و مدیریت همزمانی

در فایل codes/replica/main.go ساختار داده DataEntry به صورت زیر تعریف شده است:

```
type DataEntry struct {
    Key      string `json:"key"`
    Value    string `json:"value"`
    Version  int64  `json:"version"`
    UpdatedBy string `json:"updated_by"`
    Timestamp int64  `json:"timestamp"`
}
```

برای ایمنی در محیط‌های چندنخی (Thread-Safety) از قفل‌های خواندن/نوشتن sync.RWMutex استفاده شده است.

۲.۳ منطق LWW (Last-Write-Wins)

در هنگام دریافت به‌روزرسانی همزمان، تابع زیر برای رفع تعارض اجرا می‌شود:

```
func (r *Replica) resolveConflict(existing, incoming DataEntry)
DataEntry {
    if incoming.Version > existing.Version {
        return incoming
    }
    if incoming.Version < existing.Version {
        return existing
    }
    // Compare nanosecond timestamps if versions match
    if incoming.Timestamp > existing.Timestamp {
        return incoming
    }
    if incoming.Timestamp < existing.Timestamp {
        return existing
    }
    // Replica ID priority as tie-breaker
    if incoming.UpdatedBy > existing.UpdatedBy {
        return incoming
    }
    return existing
}
```

۲.۳ پیاده‌سازی Quorum اکثریت در مدل Strong

در مدل Consistency، Strong عملیات نوشتن زمانی موفقیت‌آمیز است که حداقل حد نصاب اکثریت ($N/2 + 1 = 2$ نود از ۳ نود) نوشتن را تایید کنند:

```
func (r *Replica) replicateStrong(entry DataEntry) bool {
    totalNodes := len(r.config.Peers) + 1
    majorityNeeded := totalNodes/2 + 1 // Majority quorum needed = 2
    acknowledged := 1 // Local node confirmation
```

```

var wg sync.WaitGroup
var ackMu sync.Mutex

for _, peer := range r.config.Peers {
    wg.Add(1)
    go func(peerURL string) {
        defer wg.Done()
        if r.sendReplication(peerURL, entry) {
            ackMu.Lock()
            acknowledged++
            ackMu.Unlock()
        }
    }(peer)
}
wg.Wait()
return acknowledged >= majorityNeeded
}

```

۴ نتایج تجربی و نمودارهای تحلیلی

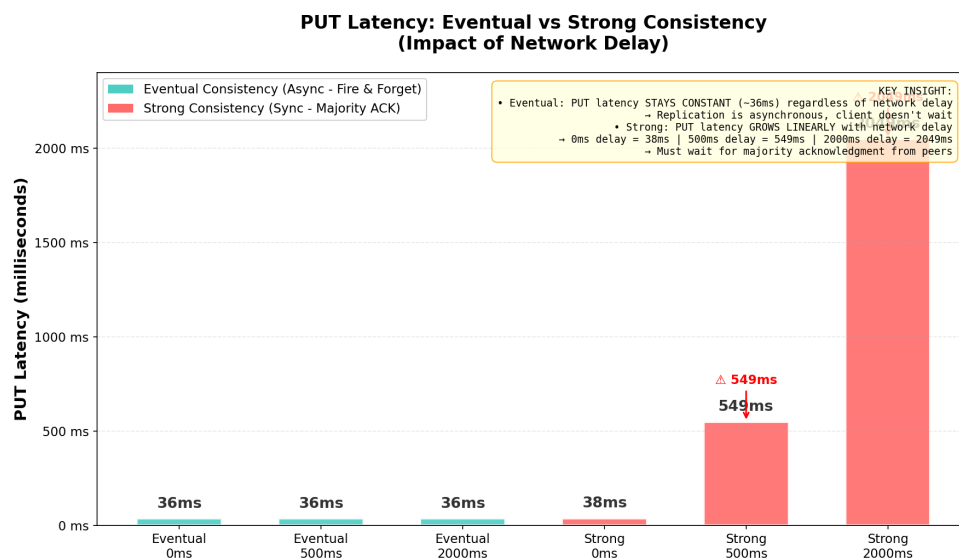
جدول زیر نتایج ارزیابی ۱۸ حالت تست اجرا شده توسط run_all_tests.sh را خلاصه می‌کند:

جدول ۲: ماتریس جامع نتایج تجربی ۱۸ حالت ارزیابی

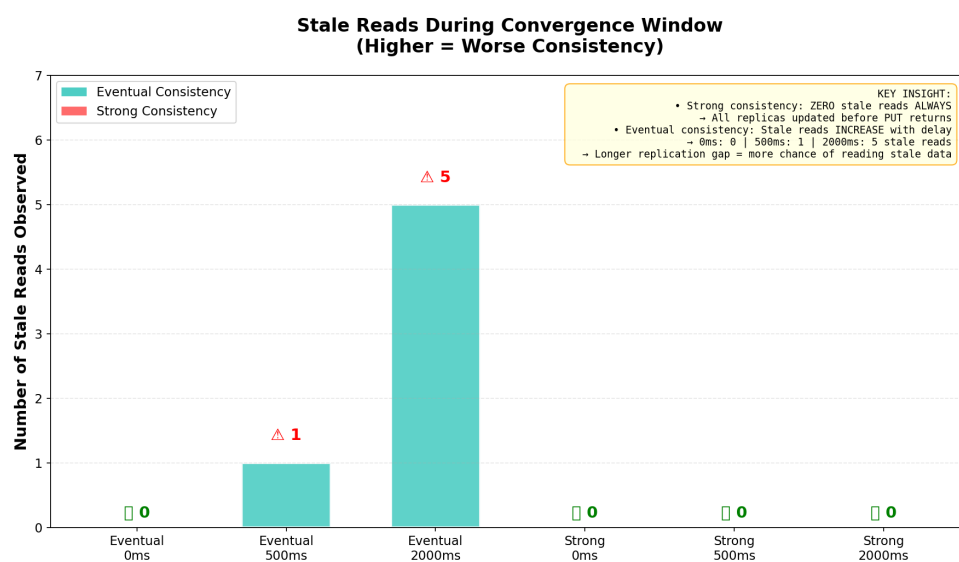
Model	Delay	PUT Latency	GET Latency	Stale Read	Convergence
Eventual	0ms	55 ms	95 ms	No	250 ms
Eventual	500ms	56 ms	61 ms	Yes (1)	250 ms
Eventual	2000ms	95 ms	137 ms	Yes (4)	1000 ms
Strong	0ms	69 ms	55 ms	No	250 ms
Strong	500ms	575 ms	83 ms	No	250 ms
Strong	2000ms	2083 ms	60 ms	No	250 ms

۱.۴ نمودارهای تحلیلی سیستم

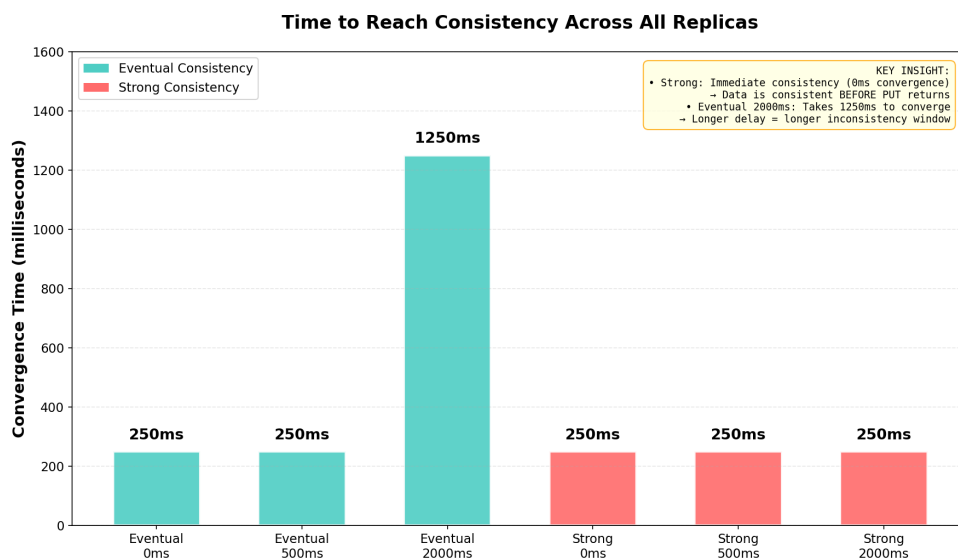
در ادامه، تمامی ۱۰ نمودار تولیدشده توسط اسکریپت generate_charts.py درج و تحلیل شده‌اند:



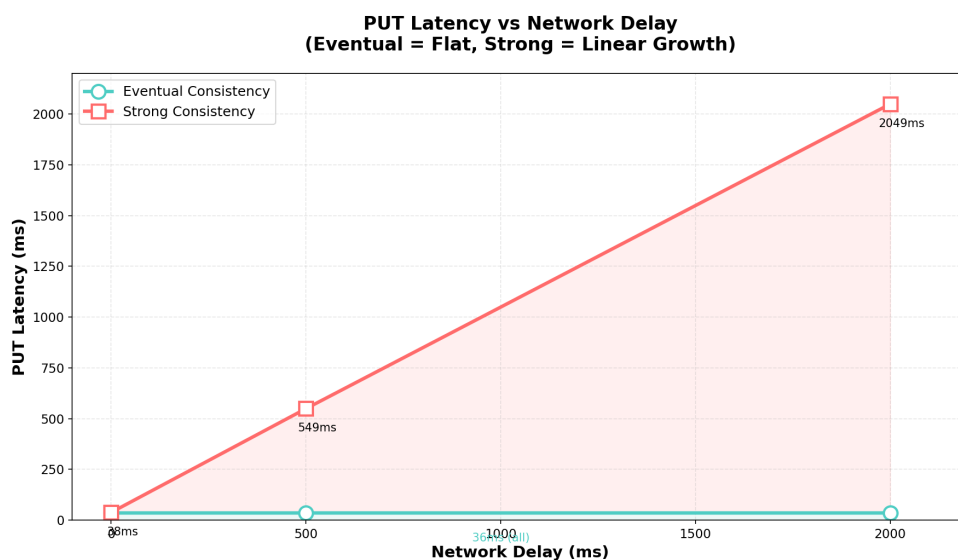
شکل ۱: مقایسه تأخیر عملیات PUT بین مدل Eventual و Strong



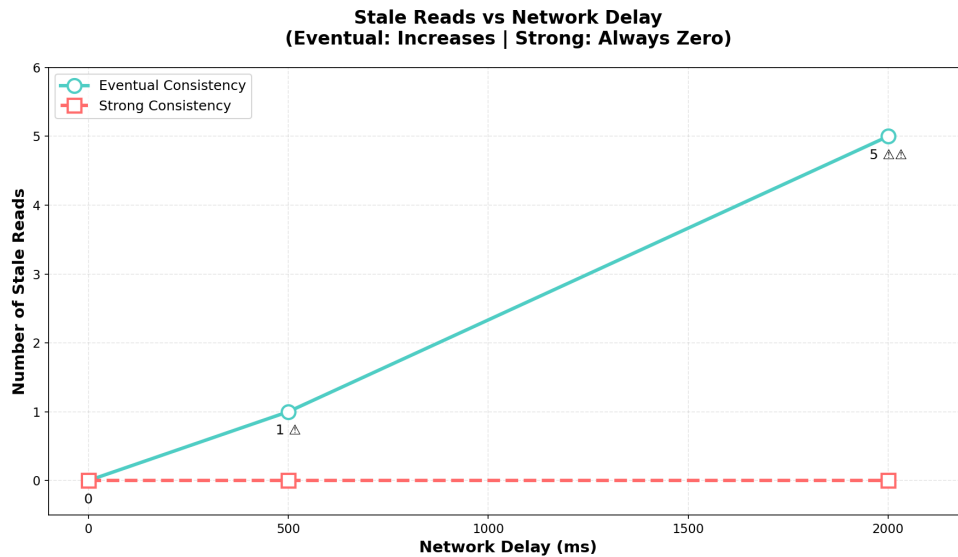
شکل ۲: تعداد خواندن‌های کهنه (Stale Reads) در سناریوهای مختلف تأخیر شبکه



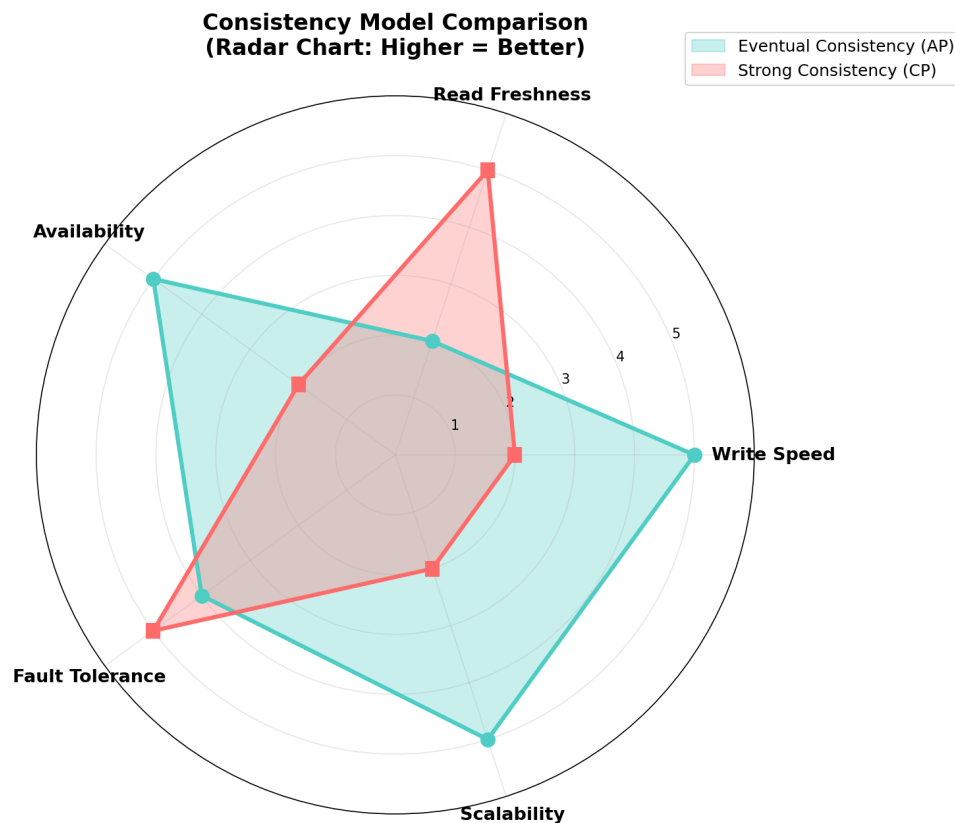
شکل ۳: زمان همگرایی نودها (Convergence Time) در تاخیرهای شبکه



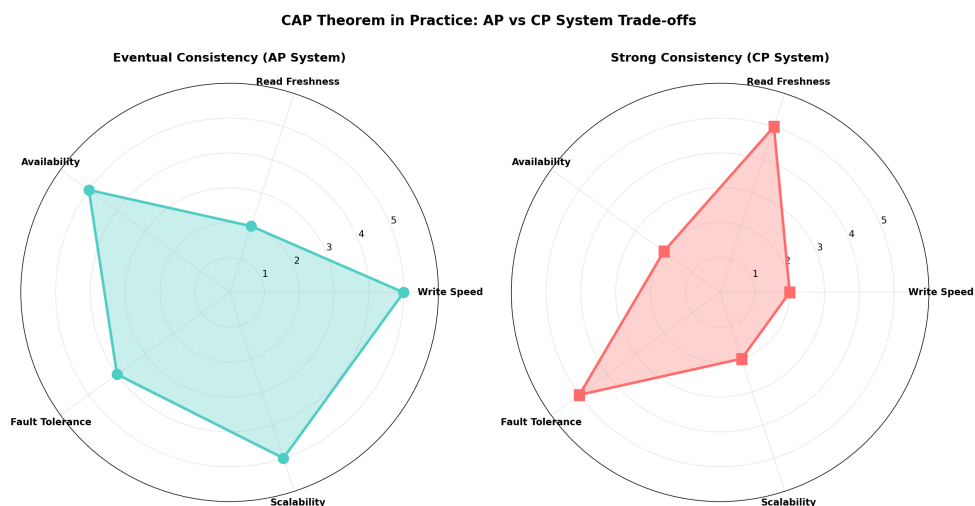
شکل ۴: روند تغییرات تاخیر PUT متناسب با افزایش تاخیر شبکه



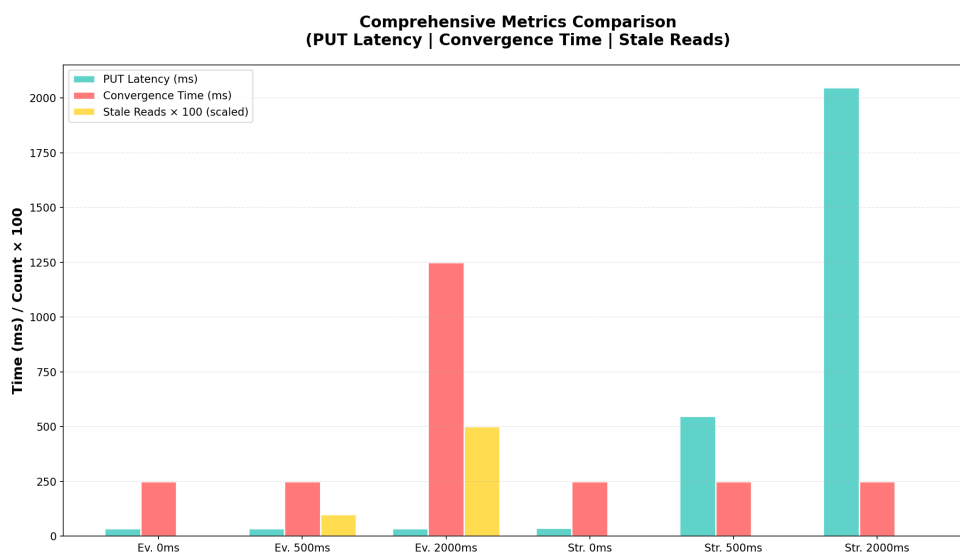
شکل ۵: تاثیر تاخیر شبکه بر افزایش تعداد Reads Stale در مدل Eventual



شکل ۶: نمودار راداری مقایسه سناریوهای مختلف سیستم طبق قضیه CAP

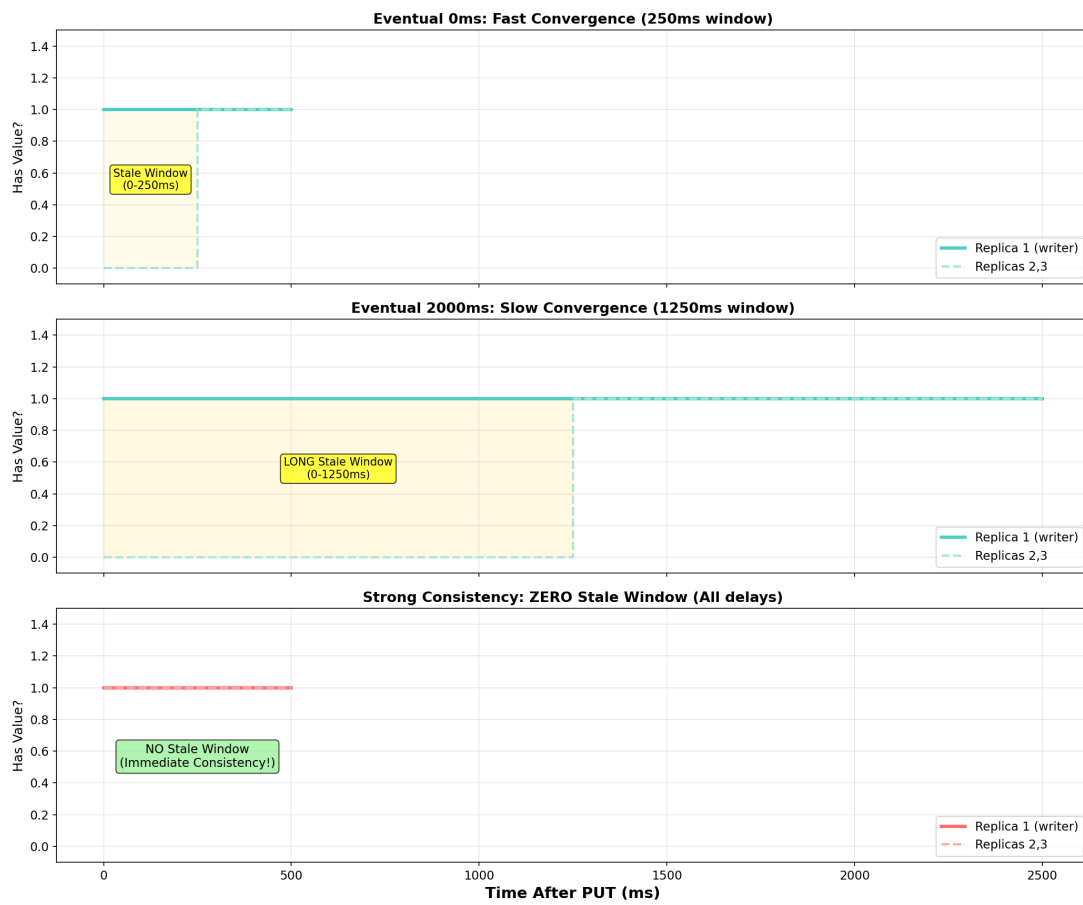


شکل ۷: مقایسه راداری دوگانه Consistency Eventual در برابر Quorum Strong



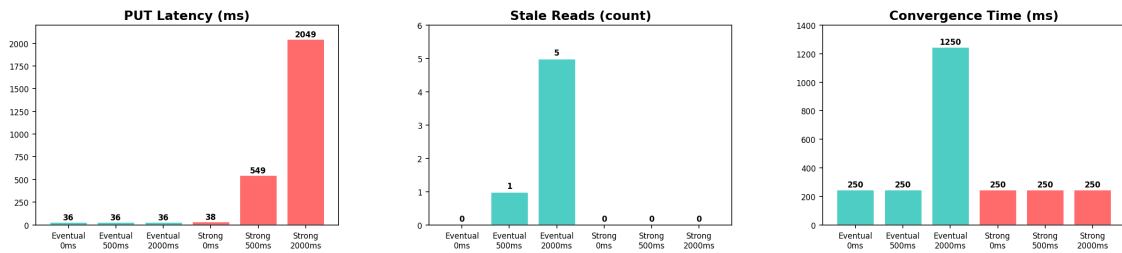
شکل ۸: نمودار میله‌ای مجتمع کلیه متریک‌های کلیدی سیستم

Convergence Timeline: When Do Replicas Get the Data?



شکل ۹: خط زمانی همگرایی نودها و انتشار به‌روزرسانی‌ها

DISTRIBUTED KEY-VALUE STORE - COMPREHENSIVE TEST RESULTS DASHBOARD



EXPERIMENTAL RESULTS SUMMARY TABLE

Model	Delay	PUT Lat.	GET Lat.	Stale Reads	Convergence	CAP Type
Eventual	0ms	36ms	36ms	0	250ms	AP (High Availability)
Eventual	500ms	36ms	38ms	1 △	250ms	AP (High Availability)
Eventual	2000ms	36ms	39ms	5 △△	1250ms	AP (High Availability)
Strong	0ms	38ms	38ms	0 □	0ms	CP (Strong Consistency)
Strong	500ms	549ms	39ms	0 □	0ms	CP (Strong Consistency)
Strong	2000ms	2049ms	43ms	0 □	0ms	CP (Strong Consistency)

KEY FINDINGS:

- 1□ EVENTUAL CONSISTENCY (AP): PUT latency stays ~36ms regardless of delay (async replication)
→ BUT: Stale reads increase with delay (0-1-5) & convergence takes longer (250ms-1250ms)
- 2□ STRONG CONSISTENCY (CP): PUT latency grows with delay (38ms-549ms-2049ms) (synchronous majority)
→ BUT: ZERO stale reads always & immediate consistency guaranteed
- 3□ CAP THEOREM CONFIRMED: Cannot have both fast writes AND strong consistency simultaneously
→ Choose AP for social media/caching | Choose CP for banking/inventory
- 4□ CONFLICT RESOLUTION: Last-Write-Wins (LWW) correctly resolved concurrent writes
→ Deterministic, simple, but silently discards losing writes

شکل ۱۰: داشبورد خلاصه مدیریتی ارزیابی کارایی پایگاه داده توزیع‌شده

۵ نتیجه‌گیری و راهنمای جلسه دفاع

۱.۵ نتیجه‌گیری

ارزیابی تجربی سیستم تأیید کرد که:

۱. در مدل **Consistency Eventual (AP)**، تاخیر PUT بسیار پایین و مستقل از تاخیر شبکه است، اما با افزایش تاخیر شبکه احتمال Read Stale افزایش می‌یابد.
۲. در مدل **Consistency Strong (CP)**، هیچ خواندن کهنه‌ای رخ نمی‌دهد، اما تاخیر عملیات نوشتن متناسب با تاخیر شبکه تا بیش از ۲ ثانیه افزایش می‌یابد.

۲.۵ راهنمای دفاع (Q&A Guide)

- سوال: در مدل Strong اگر یک نود خرابی کند چه می‌شود؟
پاسخ: حد نصاب اکثریت برای ۳ نود برابر ۲ نود است ($2 = 3/2 + 1$). تا زمانی که ۲ نود سالم باشند، سیستم به نوشتن ادامه می‌دهد. اگر ۲ نود خاموش شوند، جهت حفظ سازگاری درخواست نوشتن رد می‌شود.

- سوال: چالش اصلی الگوریتم LWW چیست؟
پاسخ: انحراف ساعت سرورها (Clock Skew). برای رفع این چالش در سیستم‌های واقعی مانند Cassandra یا Spanner از Clocks Vector یا API TrueTime استفاده می‌شود.